



Questões Práticas

Instruções: resolva as questões utilizando a linguagem Java. As soluções devem utilizar os conceitos de Orientação a Objetos trabalhados em sala de aula, como classes, atributos, métodos, construtores e encapsulamento. Também devem ser utilizados, quando solicitado, recursos como `List`, `ArrayList`, `Set`, `HashSet`, `Map`, `HashMap`, `Generics` e métodos da classe `String`.

1. Cadastro de Alunos com List

Crie um programa em Java para cadastrar alunos de uma turma.

O sistema deve possuir uma classe chamada `Aluno`, contendo os seguintes atributos:

```
private String nome;  
private String matricula;  
private double nota;
```

A classe deve possuir um construtor para receber os dados do aluno. O nome não pode ser nulo ou vazio, e deve ser armazenado sem espaços no início ou no fim. A matrícula também não pode ser vazia.

No método `main`, crie uma lista de alunos utilizando:

```
List<Aluno> alunos = new ArrayList<>();
```

Adicione pelo menos três alunos na lista e percorra a coleção exibindo os dados de cada aluno.

Requisitos:

- Criar a classe `Aluno`;
- Utilizar atributos privados;
- Criar construtor para inicializar os dados;
- Validar nome e matrícula;
- Utilizar `List<Aluno>` e `ArrayList`;
- Percorrer a lista e exibir os alunos cadastrados.

2. Códigos de Produtos com Set

Crie um programa em Java para armazenar códigos de produtos de uma loja.

O sistema deve possuir uma classe chamada `Produto`, contendo os atributos:

```
private String nome;  
private String codigo;
```

O nome do produto deve ser armazenado sem espaços no início ou no fim e em letras maiúsculas. O código também não pode ser nulo ou vazio.

No método `main`, crie uma coleção do tipo `Set<String>` para armazenar os códigos dos produtos. Adicione pelo menos cinco códigos, incluindo códigos repetidos, e mostre que o `Set` não permite duplicidade.

Requisitos:

- criar a classe `Produto`;
- utilizar atributos privados;
- utilizar métodos da classe `String`, como `trim()` e `toUpperCase()`;
- utilizar `Set<String>` e `HashSet`;
- adicionar códigos repetidos;
- exibir os códigos cadastrados sem repetição.

3. Cadastro de Clientes com Validação de String

Crie um programa em Java para cadastrar clientes.

O sistema deve possuir uma classe chamada `Cliente`, contendo os atributos:

```
private String nome;  
private String cpf;  
private String email;
```

O nome não pode ser nulo ou vazio. O CPF deve ser recebido podendo conter ponto e traço, mas deve ser armazenado apenas com números. Para isso, utilize `replace()`. Após remover os caracteres, o CPF deve possuir exatamente 11 caracteres.

O e-mail deve ser armazenado em letras minúsculas e deve conter o caractere `@`.

No método `main`, crie uma lista de clientes utilizando `List<Cliente>` e tente cadastrar clientes válidos e inválidos. Caso algum dado seja inválido, o programa deve lançar uma exceção do tipo `IllegalArgumentException` e tratar o erro com `try-catch`.

Requisitos:

- criar a classe `Cliente`;
- validar nome, CPF e e-mail;
- utilizar `trim()`, `replace()`, `toLowerCase()` e `contains()`;
- utilizar `IllegalArgumentException`;
- tratar os erros com `try-catch`;
- armazenar apenas clientes válidos em uma `List<Cliente>`;

g) Exibir os clientes cadastrados corretamente.

Exemplo de CPF recebido:

123.456.789-00

CPF armazenado:

12345678900

4. Cadastro de Veículos com Map

Crie um programa em Java para cadastrar veículos.

O sistema deve possuir uma classe chamada *Veiculo*, contendo os atributos:

```
private String placa;  
private String modelo;  
private int ano;
```

A placa deve ser armazenada sem espaços e em letras maiúsculas. O sistema deve validar se a placa possui exatamente 7 caracteres.

No método *main*, utilize um *Map<String, Veiculo>*, em que a chave será a placa do veículo e o valor será o objeto *Veiculo*.

Cadastre pelo menos três veículos. Depois, realize uma busca pela placa e exiba os dados do veículo encontrado. Caso a placa informada não exista, exiba uma mensagem informando que o veículo não foi localizado.

Requisitos:

- Criar a classe *Veiculo*;
- Validar a placa do veículo;
- Utilizar *trim()* e *toUpperCase()*;
- Utilizar *Map<String, Veiculo>* e *HashMap*;
- Usar a placa como chave do *Map*;
- Buscar um veículo pela placa;
- Exibir mensagem caso a placa não seja encontrada.

5. Sistema de Gerenciamento de Turma

Crie um sistema simples para gerenciar uma turma de alunos.

O sistema deve possuir uma classe chamada *Aluno*, contendo:

```
private String nome;  
private String matricula;  
private double nota;
```

Também deve possuir uma classe chamada **Turma**, responsável por armazenar e gerenciar os alunos.

A classe **Turma** deve utilizar uma `List<Aluno>` para armazenar os alunos cadastrados e um `Map<String, Aluno>` para permitir a busca rápida de um aluno pela matrícula.

O sistema deve impedir o cadastro de aluno com matrícula repetida, nome vazio ou nota fora do intervalo de 0 a 10. Quando isso acontecer, deve ser lançada uma exceção do tipo `IllegalArgumentException`.

Além disso, crie um método que receba uma linha de texto no seguinte formato:

<code>Nome;Matricula;Nota</code>

Esse método deve usar `split(";")` para separar os dados e cadastrar o aluno na turma.

Por fim, crie um método chamado `gerarRelatorio()`, utilizando `StringBuilder`, para exibir todos os alunos cadastrados, suas notas e a média geral da turma.

No método `main`, teste o cadastro de alunos válidos, o cadastro de alunos inválidos, a busca por matrícula e a geração do relatório final.

Requisitos:

- Criar as classes **Aluno** e **Turma**;
- Utilizar encapsulamento;
- Utilizar `List<Aluno>` e `ArrayList`;
- Utilizar `Map<String, Aluno>` e `HashMap`;
- Impedir matrícula repetida;
- Validar nome e nota;
- Utilizar `split(";")` para separar os dados de uma `String`;
- Utilizar `StringBuilder` para gerar o relatório;
- Tratar erros com `try-catch`;
- Exibir a média geral da turma.

Exemplo de entrada:

<code>Maria;9383;8.5</code> <code>Joao;1020;7.0</code> <code>Ana;1150;9.2</code>
--

Exemplo de saída esperada:

<code>Relatorio da Turma</code> <code>-----</code> <code>Nome: Maria Matricula: 9383 Nota: 8.5</code> <code>Nome: Joao Matricula: 1020 Nota: 7.0</code> <code>Nome: Ana Matricula: 1150 Nota: 9.2</code> <code>Media geral: 8.23</code>
--